
TorchFold Long-Sequence Inference Optimization on Multi-NPU Systems

A TorchFold Long-Sequence Inference Optimization on Multi-NPU Systems

A.1 Motivation

TorchFold inference maintains dense pair representations over all token pairs. For an input of length L and pair channel size C_z , the pair tensor has size $O(L^2 C_z)$, and several Pairformer and template operators allocate temporary tensors with a comparable two-dimensional footprint. This quadratic memory behavior limits the sequence lengths that can be evaluated on a single NPU. After the pair representation is distributed, long-sequence execution can also expose communication latency, synchronization stalls, layout conversions, redundant computation, and kernel-launch overhead. Practical long-sequence inference therefore requires memory scaling and execution efficiency to be addressed together.

We develop an end-to-end multi-NPU inference path that combines row-sharded pair execution with communication-aware data flow, local-query attention, exact operator chunking, fused-attention input adaptation, fused SwiGLU execution, invariant caching, and layout-aware triangle kernels. These changes affect only the execution schedule, distributed representation, backend dispatch, and tensor lifetime; they do not modify the predictive model or introduce approximations. Each attention query retains the original key/value dependency range, triangle multiplication retains the original contraction dimension, and pair transition applies the same position-wise mapping as in the unchunked model.

A.2 Setup

TorchFold maintains a single representation $\mathbf{s} \in \mathbb{R}^{L \times C_s}$, an MSA representation $\mathbf{M} \in \mathbb{R}^{N_{\text{msa}} \times L \times C_m}$, and a pair representation

$$\mathbf{Z} \in \mathbb{R}^{L \times L \times C_z}. \quad (1)$$

The pair representation dominates memory for long inputs because both persistent activations and temporary buffers scale with L^2 . On P NPUs, the target memory behavior is to store only a row shard of the pair tensor on each NPU,

$$\mathbf{Z}^{(p)} = \mathbf{Z}[I_p, :, :] \in \mathbb{R}^{L_p \times L \times C_z}, \quad \sum_{p=1}^P L_p = L, \quad (2)$$

where I_p is the token interval assigned to NPU p and $L_p = |I_p|$. This reduces persistent pair storage from $O(L^2 C_z)$ to approximately $O(L^2 C_z / P)$ per NPU.

A.3 Parallel Execution and Performance Optimization

The long-sequence inference path combines distributed execution, fused-attention adaptation, and schedule- and kernel-level optimization. These techniques reduce memory use, communication overhead, synchronization stalls, redundant computation, and data movement without changing the model definition or introducing approximations.

Multi-NPU parallel execution. The pair representation is sharded by row across NPUs, so each rank stores and updates only its local output rows while retaining the full dependency range required by

the original computation. Row-local modules execute directly on the shard; incoming pair modules exchange row and column views; triangle multiplication streams the remote blocks required by each contraction; and MSA and diffusion attention compute local query rows against global keys and values. Pair-derived confidence outputs are gathered only at the external interface, large grid-attention and pair-transition kernels are chunked to limit temporary memory, and rank-consistent metadata keeps stochastic preprocessing synchronized. This schedule reduces persistent and temporary memory while preserving the exact Pairformer, template, attention, and confidence-head semantics.

NPU Fusion Attention input adaptation. An adapter at `fastnn.dot_product_attention` reads the runtime (B, H, Q, K) dimensions from the query and key tensors and uses the target-logit shape $[B, H, Q, K]$ to validate bias and mask broadcasting, including local-query cases where $Q \neq K$. Supported pair-bias layouts are normalized to the backend’s `pse` input, while valid-key masks are converted to compact `atten_mask` layouts without materializing large four-dimensional masks; padded local rows are zeroed after execution when required. Calls with unambiguous shape and broadcast semantics use `npu_fusion_attention`, whereas unsupported inputs remain on the standard attention path, preserving correctness through guarded dispatch.

NPU-Fused SwiGLU operator. Given a contiguous projection output arranged as $[A \mid B]$, the fused NPU operator computes $\text{SiLU}(A) \odot B$ within a single kernel. It removes the explicit SiLU intermediate tensor and the separate element-wise multiplication, thereby reducing kernel launches and activation-memory traffic while preserving the original SwiGLU computation and lowering module latency.

Communication overlap and distributed data flow. Collective latency is hidden by launching communication for the next chunk or block while computing the current one in triangle block broadcast, grid input exchange, and diffusion activation gathering. The distributed layout is also selected according to each consumer’s access pattern: attention computes local query rows with the complete key/value range, pairwise heads consume row-sharded representations, symmetric masks are derived locally, and collective results are received directly in their target layout. Together, these choices reduce exposed communication, communicated data, duplicated computation, and intermediate stacking, concatenation, transposition, and contiguous-copy operations.

Synchronization and redundant-work elimination. Host-known sequence lengths and padding structures are used to construct static attention masks before repeated Pairformer and diffusion paths, avoiding device reductions and host scalar reads that would otherwise create execution bubbles. Values that remain invariant across chunks or diffusion steps, including pair bias, pair logits, and supported data-type conversions, are computed once and reused. This removes unnecessary synchronization, repeated projections and casts, and redundant HBM traffic while allowing NPU kernels to be issued more continuously.

Triangle Multiplication Optimization. The triangle multiplication path reorganizes the small projection and gating weights once, allowing the projections to directly produce channel-first A/B operands for batched matrix multiplication. This replaces an $\mathcal{O}(N^2C)$ activation reorder with an $\mathcal{O}(C^2)$ weight reorder. Metadata-only views, in-place gating, and a fused multiply-add further reduce tensor copies, device-memory traffic, and module latency without changing the mathematical result.

Triangle Attention Optimization. The original joint query-key-value projection is divided into three narrow projections that directly generate contiguous query, key, and value tensors in batch-sequence-hidden layout. The fused attention operator consumes and returns tensors in the same layout, eliminating input head-layout transformations and the output-layout restoration. This reduces intermediate memory usage, device-memory traffic, and the combined row-and-column module latency.